Centro de Informática UFPE

Universidade Federal de Pernambuco

MaratonaCIn

Introdução à Programação Competitiva

Primeiros passos e Standard Library (STL)

rev. 1.0

CC BY NC SA

Lua Guimarães <lgf>

O que é Programação Competitiva?

Centro de Informática UFPE | Universidade Federal de Pernambuco | MaratonaCIn

Programação Competitiva é a arte de resolver problemas algorítmicos com **restrições de tempo e espaço**, geralmente em competições online ou presenciais.

O Ciclo de Resolução:

- **Modelagem:** Entender e abstrair o problema lógico.
- **Codificação:** Implementar a solução rapidamente (geralmente em C++, Python ou Java).
- **Julgamento:** Submeter o código a um *Juiz Online*, buscando o cobiçado **Accepted (AC)** antes do limite de tempo (**TLE**).

Por que competir?

- Desenvolve raciocínio algorítmico rápido e sólido
- Excelente preparação para entrevistas técnicas (*Big Techs*)
- Ensina técnicas avançadas de programação em baixo nível que diferenciam o estudante
- Desenvolve resiliência e habilidade de *debugar* sob pressão
- Estimula trabalho em equipe (em provas como Maratona SBC / ICPC)
- Comunidade global ativa e engajada

A Ferramenta: Por que C++?

Centro de Informática UFPE | Universidade Federal de Pernambuco | MaratonaCIn

O **C++** será a nossa principal linguagem. Ela é, de longe, a mais utilizada por competidores no mundo inteiro.

Principais Vantagens:

- **Performance Absoluta:** Extremamente rápida e com controle de memória eficiente, essencial para não levar *Time Limit Exceeded*.
- **O Poder da STL:** A *Standard Template Library* já traz estruturas de dados (vetores, pilhas, árvores) e algoritmos de ordenação super otimizados prontos para uso.
- **Soberania na Comunidade:** Quase 100% dos editoriais (soluções oficiais) e tutoriais avançados na internet são escritos em C++.

Plataformas de Prática

A teoria só faz sentido com muita prática. Vamos utilizar juízes online para treinar e testar nossos códigos:

- **Codeforces:** A maior plataforma do mundo. Competições frequentes que rendem *rating* (sistema de ranking).
- **CSES Problem Set:** Uma lista de problemas clássicos que serve como a "bíblia" prática para aprender os algoritmos base.
- **AtCoder:** Plataforma excelente com problemas de altíssima qualidade lógica e matemática.

Anatomia de um problema

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Ao submeter seu código, a plataforma vai testá-lo contra vários "casos de teste". Estes são os principais vereditos:

- **AC (Accepted):** O tão sonhado verde! Código correto e dentro do tempo.
- **WA (Wrong Answer):** Lógica incorreta, erro de formatação na saída ou algum caso de borda esquecido.
- **TLE (Time Limit Exceeded):** Código muito lento. Precisa otimizar.
- **RE (Runtime Error):** O programa "quebrou" (ex.: segfault, crash).
- **CE (Compile Error):** Erro de sintaxe (ex.: faltou um ponto e vírgula).

Todo problema é formado por 4 seções principais que exigem atenção:

- **TL / ML (Limites):** *Time Limit* e *Memory Limit*: as restrições de eficiência do seu código.
- **Statement (Descrição):** A historinha ou formulação matemática do que deve ser feito.
- **Input (Entrada):** O formato dos dados e os ***limites das variáveis*** (ex.: $1 \leq N \leq 10^5$). Crucial para prever a complexidade do algoritmo e evitar vazamento de memória.
- **Output (Saída):** O formato exato e rígido da resposta esperada. O juiz é cego: um espaço extra ou falta de quebra de linha pode gerar um veredito de WA.

Primeiro código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Vamos começar a montar o nosso template de programação competitiva a partir do código mais básico em qualquer linguagem: Um "Hello World"!

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

O código é formado por 4 estruturas principais:

1. **Bibliotecas (Includes):** Importamos as ferramentas do C++.
2. **Declaração e Leitura:** Criamos o espaço na memória `std::string name;` e capturamos a entrada enviada pelo juiz online usando `std::cin`.
3. **Saída (Output):** Imprimimos a resposta no formato exato exigido usando `std::cout`.
4. **Retorno de Sucesso:** O `return 0;` avisa ao sistema (e ao juiz) que o programa terminou sem erros.

Primeiro código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Vamos começar a montar o nosso template de programação competitiva a partir do código mais básico em qualquer linguagem: Um "Hello World"!

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

O código é formado por 4 estruturas principais:

1. **Bibliotecas (Includes):** Importamos as ferramentas do C++.
2. **Declaração e Leitura:** Criamos o espaço na memória `std::string name;` e capturamos a entrada enviada pelo juiz online usando `std::cin`.
3. **Saída (Output):** Imprimimos a resposta no formato exato exigido usando `std::cout`.
4. **Retorno de Sucesso:** O `return 0;` avisa ao sistema (e ao juiz) que o programa terminou sem erros.

Primeiro código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Vamos começar a montar o nosso template de programação competitiva a partir do código mais básico em qualquer linguagem: Um "Hello World"!

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

O código é formado por 4 estruturas principais:

1. **Bibliotecas (Includes):** Importamos as ferramentas do C++.
2. **Declaração e Leitura:** Criamos o espaço na memória `std::string name;` e capturamos a entrada enviada pelo juiz online usando `std::cin`.
3. **Saída (Output):** Imprimimos a resposta no formato exato exigido usando `std::cout`.
4. **Retorno de Sucesso:** O `return 0;` avisa ao sistema (e ao juiz) que o programa terminou sem erros.

Primeiro código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Vamos começar a montar o nosso template de programação competitiva a partir do código mais básico em qualquer linguagem: Um "Hello World"!

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

O código é formado por 4 estruturas principais:

1. **Bibliotecas (Includes):** Importamos as ferramentas do C++.
2. **Declaração e Leitura:** Criamos o espaço na memória `std::string name;` e capturamos a entrada enviada pelo juiz online usando `std::cin`.
3. **Saída (Output):** Imprimimos a resposta no formato exato exigido usando `std::cout`.
4. **Retorno de Sucesso:** O `return 0;` avisa ao sistema (e ao juiz) que o programa terminou sem erros.

Primeiro código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Vamos começar a montar o nosso template de programação competitiva a partir do código mais básico em qualquer linguagem: Um "Hello World"!

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

O código é formado por 4 estruturas principais:

1. **Bibliotecas (Includes):** Importamos as ferramentas do C++.
2. **Declaração e Leitura:** Criamos o espaço na memória `std::string name;` e capturamos a entrada enviada pelo juiz online usando `std::cin`.
3. **Saída (Output):** Imprimimos a resposta no formato exato exigido usando `std::cout`.
4. **Retorno de Sucesso:** O `return 0;` avisa ao sistema (e ao juiz) que o programa terminou sem erros.

Primeiro Código

```cpp
#include <iostream>
#include <string>

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

Código C++ com o prefixo `std::` em tudo. É o jeito mais explícito: deixa claro de onde cada nome vem, mas é verboso demais para usar em contest.

Primeiro Código

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    std::string name;
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;

    return 0;
}
```

`using namespace std;` importa todos os nomes do namespace `std` para o escopo global: como dizer "já sei de onde vêm as coisas, não preciso repetir `std::` o tempo todo".

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name;
    cin >> name;

    cout << "Hello, " << name << "!" << endl;

    return 0;
}
```

Com `using namespace std;`, escrevemos `string`, `cin`, `cout` diretamente. Em programação competitiva, esse é o padrão universal.

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    string name;
    cin >> name;

    cout << "Hello, " << name << "!" << endl;

    return 0;
}
```

`#include <bits/stdc++.h>` inclui **tudo** da biblioteca padrão de uma vez: `<vector>`, `<string>`, `<algorithm>`, tudo.

É uma extensão do **GCC** – não faz parte do padrão C++ e não funciona no MSVC nem no Clang sem configuração extra. Em CP isso não importa: os juízes online usam GCC, então economiza tempo e garante que nenhum include vai faltar.

Primeiro Código


```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    string name;
    cin >> name;

    cout << "Hello, " << name << "!" << endl;

    return 0;
}
```

Por padrão, `cin` / `cout` são sincronizados com `scanf` / `printf` do C, o que tem um **alto custo de performance**. A cada operação, o programa sincroniza os buffers de ambas as bibliotecas.

- `ios::sync_with_stdio(false)` desativa essa sincronização
- `cin.tie(0)` desvincula `cin` de `cout`, eliminando flushes desnecessários

Com $n \geq 10^5$ linhas de input, sem isso → **TLE**.

Primeiro Código


```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    string name;
    cin >> name;

    cout << "Hello, " << name << "!" << '\n';

    return 0;
}
```

`endl` faz duas coisas: insere `\n` **e** força um flush do buffer de saída. Em loops com muitas escritas, isso é devastador para a performance.

Use sempre `'\n'`, o buffer é liberado automaticamente no final do programa.

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    int tc; cin >> tc; while (tc--) {
        string name;
        cin >> name;

        cout << "Hello, " << name << "!" << '\n';
    }

    return 0;
}
```

Muitos problemas têm múltiplos casos de teste: a primeira linha informa quantos são (`tc`), e o código processa cada um no loop. O `while (tc--)` decrementa `tc` e continua enquanto for positivo: padrão universal em CP.

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    int tc; cin >> tc; while (tc--) {
        string name;
        cin >> name;

        cout << "Hello, " << name << "!" << '\n';
    }

    return 0;
}
```

Para testar localmente sem colar input no terminal, `freopen` redireciona `stdin` / `stdout` para arquivos. Antes de submeter, basta remover essas duas linhas: o juiz online usa `stdin` / `stdout` normalmente.

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll  = long long;
using pii = pair<int, int>;
using vi  = vector<int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    int tc; cin >> tc; while (tc--) {
        string name;
        cin >> name;

        cout << "Hello, " << name << "!" << '\n';
    }

    return 0;
}
```

Aliases de tipo eliminam ruído visual no código:

- `ll` no lugar de `long long`
- `pii` no lugar de `pair<int,int>`
- `vi` no lugar de `vector<int>`

São definidos com `using`, a sintaxe moderna do C++11: equivalente ao `typedef`, mas mais legível.

Primeiro Código

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll  = long long;
using pii = pair<int, int>;
using vi  = vector<int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout << fixed << setprecision(10);

    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    int tc; cin >> tc; while (tc--) {
        string name;
        cin >> name;

        cout << "Hello, " << name << "!" << '\n';
    }

    return 0;
}
```

`fixed` + `setprecision(N)` configura o `cout` para imprimir sempre `N` casas decimais em notação fixa. Sem isso, problemas com `double` podem gerar **WA** por diferença de arredondamento na saída.

`10` casas decimais é o suficiente para a maioria dos problemas.

Centro de Informática UFPE

Universidade Federal de Pernambuco

MaratonaCIn

Complexidade Temporal e Espacial

Entendendo o BIG O

Complexidade Temporal e Espacial

Em competições, não basta ter um código que funciona – ele precisa ser rápido o suficiente para rodar dentro do tempo limite (1–2s). A análise de complexidade nos ajuda a **prever a velocidade de um algoritmo** sem precisar executá-lo.

Regra prática: CPUs modernas executam cerca de 10^8 operações por segundo. Se sua complexidade ultrapassa esse limite para o N do problema, o veredicto provavelmente será **TLE**.

A **notação Big O** descreve como o tempo de execução cresce conforme N aumenta. Focamos no **pior caso** e ignoramos constantes e termos de menor ordem.

Exemplo: $3N^2 + 5N + 10 \Rightarrow O(N^2)$

Complexidades comuns em CP:

- $O(1)$: acesso a array, operação aritmética
- $O(\log N)$: busca binária, operações em heap
- $O(N)$: loop simples, leitura do input
- $O(N \log N)$: ordenação (`std::sort`)
- $O(N^2)$: dois loops aninhados, força bruta simples
- $O(2^N)$: geração de todos os subconjuntos
- $O(N!)$: geração de todas as permutações

Complexidade Temporal e Espacial

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

Complexidade	N máx. (≈1s)	N = 10	N = 100	N = 10^3	N = 10^4	N = 10^5	N = 10^6
O(log N)	≈ 10^{10}	~3	~7	~10	~13	~17	~20
O(N)	≈ 10^8	10	100	1.000	10.000	100.000	1.000.000
O(N log N)	≈ 10^6	~33	~664	~9.965	~132.877	~1.661.000	~19.932.000
O(N^2)	≈ 10^4	100	10.000	10^6	10^8	10^{10}	10^{12}
O(N^3)	≈ 4×10^2	1.000	10^6	10^9	10^{12}	10^{15}	10^{18}
O(2^N)	≈ 26	1.024	≈ 10^{30}	∞	∞	∞	∞
O(N!)	≈ 11	3.628.800	≈ 10^{157}	∞	∞	∞	∞

Dentro do limite · Provável TLE

Valores em número de operações. Assumindo ≈ 10^8 ops/s no juiz.

Complexidade Espacial

Assim como o tempo, a **memória é um recurso limitado**. Em CP, o limite típico é 256–512 MB – estourá-lo gera **MLE** (*Memory Limit Exceeded*).

Complexidade de Espaço mede quanta memória o programa utiliza em função de N.

Para calcular a memória de uma estrutura, basta multiplicar o número de elementos pelo tamanho do tipo.

Exemplo: um array de 10^6 `int`s usa:

$$10^6 \times 4\,\text{bytes} = 4\,\text{MB}$$

Cabe confortavelmente em 256 MB. O mesmo array com `long long` usa o dobro: **8 MB**.

Escolher o tipo de dado correto é o primeiro passo para controlar o uso de memória. Cada tipo tem um tamanho fixo e um intervalo de valores que define quando deve ser usado.

Tipos mais usados em CP:

- `int` – uso geral; suficiente para $|a| \leq 2 \times 10^9$
- `long long` – quando o resultado pode ultrapassar 2×10^9
- `double` – respostas com casas decimais (~15 dígitos de precisão)
- `char` – processamento de caracteres e strings
- `bool` – arrays de flags e estado binário

Complexidade Espacial

Tipo	O que armazena	Tamanho	Intervalo de Valores
int	Inteiro	4 bytes	$\approx \pm 2 \times 10^9$
long long (ll)	Inteiro 64-bit	8 bytes	$\approx \pm 9 \times 10^{18}$
char	Caractere	1 byte	ASCII (0–127)
bool	Booleano	1 byte	true / false
float	Real precisão simples	4 bytes	~7 dígitos decimais
double	Real precisão dupla	8 bytes	~15 dígitos decimais
long double	Real precisão estendida	16 bytes	~18–19 dígitos decimais

Cada tipo de dado armazena um tipo diferente de informação e ocupa uma quantidade específica de memória.

Centro de Informática UFPE

Universidade Federal de Pernambuco

MaratonaCIn

Biblioteca Padrão (STL)

Containers e algoritmos

Biblioteca Padrão (STL)

A STL é uma coleção de estruturas de dados e algoritmos prontos, testados e eficientes. Usá-la economiza tempo e evita erros.

Você pode importá-la completamente usando:

```
#include <bits/stdc++.h>
```

Em juízes de CP isso está sempre disponível. Em produção, prefira incluir apenas os headers necessários.

Containers – estruturas de dados prontas:

- `vector`, `string`, `array`
- `stack`, `queue`, `deque`
- `set`, `multiset`
- `map`, `unordered_map`
- `priority_queue`

Algoritmos – funções sobre containers:

- `sort`, `reverse`, `unique`
- `lower_bound`, `upper_bound`, `binary_search`
- `min`, `max`, `min_element`, `max_element`
- `count`, `find`, `next_permutation`

Array dinâmico `vector<T>`

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

O que é? Um array dinâmico que cresce e diminui de tamanho automaticamente.

Quando usar? Quase sempre! É a estrutura mais flexível e usada em CP.

Observação: `string` é essencialmente um `vector<char>` com operações extras.

Operação	Custo
`push_back(x)`	$O(1)$ amortizado
`pop_back()`	$O(1)$
`v[i]`	$O(1)$
`size()`	$O(1)$
`insert` / `erase` (meio)	$O(N)$

```cpp
vector<int> v;

v.push_back(10);
v.push_back(20);
v.push_back(30);
// v = [10, 20, 30]

cout << v[0];      // 10
cout << v.size();  // 3
cout << v.back();  // 30

// Inicializar com tamanho e valor padrão
vector<int> zeros(5, 0); // [0, 0, 0, 0, 0]

// Range-based for
for (int x : v)
    cout << x << '\n';

// Remover último elemento
v.pop_back(); // v = [10, 20]

// Ordenar
sort(v.begin(), v.end());
```

Strings `string`

O que é? Uma sequência mutável de caracteres com operações de texto embutidas.

Quando usar? Processamento de texto, parsing de entrada, comparação e manipulação de strings.

Dica: `find()` retorna `string::npos` se não encontrar — sempre compare com ele.

Operação	Custo
`s[i]`, `s.back()`	$O(1)$
`s.size()`	$O(1)$
`s + t` (concat)	$O(N)$
`s.substr(pos, len)`	$O(N)$
`s.find(sub)`	$O(N \cdot M)$

```
string s = "hello";

s += " world";            // "hello world"
cout << s.size();         // 11
cout << s[0];             // 'h'
cout << s.back();         // 'd'

// Substring
cout << s.substr(6, 5);   // "world"

// Busca — string::npos se não achar
size_t pos = s.find("world");
if (pos ≠ string::npos)
    cout << pos;          // 6

// Iterar por caracteres
for (char c : s) cout << c;

// Converter
string n = to_string(42); // "42"
int x    = stoi("123");   // 123
```

Array estático `array<T, N>`

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

O que é? Wrapper sobre array C com tamanho **fixo em tempo de compilação**.

Quando usar? Quando o tamanho é constante e conhecido – mais seguro que `T arr[N]` pois tem `.size()`, `.begin()` e `.end()`.

Observação: Não cresce. Para tamanho dinâmico, use `vector<T>`.

Operação	Custo
`a[i]`, `a.at(i)`	$O(1)$
`a.size()`	$O(1)$
`a.front()`, `a.back()`	$O(1)$
`a.fill(val)`	$O(N)$
`sort(a.begin(), a.end())`	$O(N \log N)$

```cpp
array<int, 5> a = {3, 1, 4, 1, 5};

cout << a[0];         // 3
cout << a.size();     // 5
cout << a.front();    // 3
cout << a.back();     // 5

// Preencher com valor padrão
array<int, 5> zeros;
zeros.fill(0);        // {0, 0, 0, 0, 0}

// Sort funciona normalmente
sort(a.begin(), a.end());
// a = {1, 1, 3, 4, 5}

// Iterar
for (int x : a) cout << x << ' ';

// Passar para função sem perder o tamanho
void f(array<int, 5>& arr) {
    cout << arr.size(); // 5
}
```

N-uplas `pair<A,B>` `tuple<...>`

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

`pair<A, B>`: Par de dois valores; suporta comparação lexicográfica.

`tuple<A, B, C, ...>`: Generalização do pair para N valores.

Quando usar? Agrupar valores sem criar struct; retornar múltiplos valores; ordenar por critério composto.

	Acesso
`pair`	`.first`, `.second`
`tuple`	`get<i>(t)`
C++17	`auto [a, b] = p`

```
pair<int, int> p = {3, 7};
cout << p.first;   // 3
cout << p.second;  // 7

// Comparação lexicográfica
pair<int,int> a = {1, 5}, b = {1, 3};
cout << (a > b);   // true (1==1, então 5>3)

// Ordenar por segundo elemento
vector<pair<int,int>> v = {{1,3},{2,1},{0,2}};
sort(v.begin(), v.end()); // ordena pelo .first

// tuple
tuple<int, string, double> t = {1, "abc", 3.14};
cout << get<0>(t); // 1
cout << get<1>(t); // abc

// Structured bindings (C++17)
auto [x, y] = p;
auto [id, nome, nota] = t;
```

Pilha `stack<T>`

O que é? Pilha LIFO – o último elemento inserido é o primeiro a sair.

Quando usar? DFS iterativo, verificação de parênteses balanceados, avaliação de expressões.

Observação: Não tem iteradores – só acesso ao topo com `top()`.

Operação	Custo
`push(x)`	$O(1)$
`pop()`	$O(1)$
`top()`	$O(1)$
`empty()`, `size()`	$O(1)$

```
stack<int> s;

s.push(10);
s.push(20);
s.push(30);

cout << s.top();  // 30 (topo da pilha)
cout << s.size(); // 3

s.pop();          // remove o topo
cout << s.top();  // 20

s.pop();
cout << s.top();  // 10

// Esvaziar (não tem iterador)
while (!s.empty()) {
    cout << s.top() << ' '; // 10
    s.pop();
}
```

Fila `queue<T>`

O que é? Fila FIFO – o primeiro elemento inserido é o primeiro a sair.

Quando usar? BFS (Busca em Largura), processamento de tarefas em ordem de chegada.

Observação: Não tem iteradores – só acesso à frente (`front()`) e ao final (`back()`).

Operação	Custo
`push(x)`	$O(1)$
`pop()`	$O(1)$
`front()`	$O(1)$
`back()`	$O(1)$
`empty()` , `size()`	$O(1)$

```cpp
queue<int> q;

q.push(10);
q.push(20);
q.push(30);

cout << q.front(); // 10 (primeiro a entrar)
cout << q.back();  // 30 (último a entrar)
cout << q.size();  // 3

q.pop();           // remove o front
cout << q.front(); // 20

q.pop();
cout << q.front(); // 30

// Processar todos em ordem de chegada
while (!q.empty()) {
    cout << q.front() << ' '; // 30
    q.pop();
}
```

Fila dupla `deque<T>`

O que é? Fila duplamente terminada — inserção e remoção em $O(1)$ nas **duas extremidades**.

Quando usar? Sliding window monotônico, quando precisa de push/pop em ambas as pontas.

Observação: Mais lento que `vector` para acesso aleatório e iteração, mas suporta `push_front` e `pop_front` em $O(1)$.

Operação	Custo
`push_back(x)` / `push_front(x)`	$O(1)$
`pop_back()` / `pop_front()`	$O(1)$
`d[i]`	$O(1)$
`size()`	$O(1)$

```cpp
deque<int> d;

d.push_back(10);   // d = [10]
d.push_back(20);   // d = [10, 20]
d.push_front(5);   // d = [5, 10, 20]

cout << d.front(); // 5
cout << d.back();  // 20
cout << d.size();  // 3
cout << d[1];      // 10 (acesso por índice)

d.pop_front();     // remove da frente → d = [10, 20]
d.pop_back();      // remove do fim    → d = [10]

// Iterar normalmente
for (int x : d) cout << x << ' ';
```

Conjuntos `set<T>` `multiset<T>`

`set<T>`: Conjunto ordenado de elementos **únicos** (árvore rubro-negra internamente).

`multiset<T>`: Igual, mas permite elementos repetidos.

Quando usar? Manter elementos únicos e ordenados; verificar existência, inserir e remover em $O(\log N)$.

Operação	Custo
`insert(x)`	$O(\log N)$
`erase(x)`	$O(\log N)$
`find(x)`	$O(\log N)$
`count(x)`	$O(\log N)$
`lower_bound(x)`	$O(\log N)$

```cpp
set<int> s = {3, 1, 4, 1, 5, 9};
// s = {1, 3, 4, 5, 9}  (único + ordenado)

s.insert(2);         // s = {1, 2, 3, 4, 5, 9}
s.erase(3);          // s = {1, 2, 4, 5, 9}

cout << s.count(4); // 1 (existe)
cout << s.count(7); // 0 (não existe)

// Primeiro elemento ≥ 4
auto it = s.lower_bound(4);
cout << *it;         // 4

// Iterar em ordem crescente
for (int x : s) cout << x << ' ';

// multiset: permite repetições
multiset<int> ms = {1, 1, 2, 3};
ms.erase(ms.find(1)); // remove só UMA ocorrência de 1
// ms = {1, 2, 3}
```

Dict `map<K,V>` `unordered_map<K,V>`

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

`map<K, V>`: Pares chave-valor ordenados por chave. $O(\log N)$ por operação.

`unordered_map<K, V>`: Hash table. $O(1)$ médio, sem ordenação.

Regra geral: Prefira `unordered_map` para lookup puro; use `map` quando precisar de ordenação ou `lower_bound`.

Operação	`map`	`unordered_map`
`[key]`, `find`	$O(\log N)$	$O(1)$ médio
`insert`, `erase`	$O(\log N)$	$O(1)$ médio
Ordem garantida	Sim	Não

```cpp
map<string, int> freq;
freq["ana"]++;
freq["bob"]++;
freq["ana"]++;

cout << freq["ana"]; // 2

// Iterar em ordem de chave
for (auto& [k, v] : freq)
    cout << k << ": " << v << '\n';

// Verificar existência sem criar entrada
if (freq.count("bob"))
    cout << "existe\n";

// Cuidado: [] cria a chave se não existir!
// Use .find() ou .count() para checar antes

// unordered_map — mesma API, mais rápido
unordered_map<int, int> mp;
mp[1] = 100;
mp[2] = 200;
cout << mp[1]; // 100
```

Fila prioritária `priority_queue<T>`

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

O que é? Fila de prioridade – sempre acessa o **maior** elemento em $O(1)$.

Internamente: max-heap (árvore binária de máximo).

Quando usar? Dijkstra, encontrar o k-ésimo maior, processamento guloso por prioridade.

Operação	Custo
`push(x)`	$O(\log N)$
`pop()`	$O(\log N)$
`top()`	$O(1)$
`empty()`, `size()`	$O(1)$

```
// Max-heap (padrão)
priority_queue<int> pq;
pq.push(5); pq.push(1); pq.push(8);

cout << pq.top(); // 8
pq.pop();
cout << pq.top(); // 5

// Min-heap — inverter o comparador
priority_queue<int, vector<int>,
                    greater<int>> minpq;

minpq.push(5); minpq.push(1); minpq.push(3);

cout << minpq.top(); // 1

// Com pares — ordena pelo primeiro elemento
priority_queue<pair<int, int>> pq2;
pq2.push({10, 1});
pq2.push({5,  2});

cout << pq2.top().first; // 10
```

Algoritmos – Ordenação

Os algoritmos da STL operam em **ranges** – pares de iteradores `(begin, end)` que definem o intervalo de atuação.

Algoritmo	O que faz	Custo
`sort(b, e)`	Ordena crescente	$O(N \log N)$
`sort(b, e, cmp)`	Ordena com comparador	$O(N \log N)$
`reverse(b, e)`	Inverte o range	$O(N)$
`unique(b, e)`	Remove adj. duplicados	$O(N)$
`next_permutation`	Próxima permutação lexicográfica	$O(N)$

Dica: Para remover **todos** os duplicados de um vector, use `sort` + `unique` + `erase`.

```cpp
vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

// Ordenar crescente
sort(v.begin(), v.end());
// v = {1, 1, 2, 3, 4, 5, 6, 9}

// Remover duplicatas
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
// v = {1, 2, 3, 4, 5, 6, 9}

// Ordenar decrescente
sort(v.begin(), v.end(), greater<int>());

// Comparador customizado
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b;
});

// Gerar todas as permutações
string s = "abc";
do {


    cout << s << '\n';
```

Algoritmos – Busca e Utilidades

Algoritmo	O que faz	Custo
`lower_bound`	Primeiro ≥ x	$O(\log N)$
`upper_bound`	Primeiro > x	$O(\log N)$
`binary_search`	Existe x?	$O(\log N)$
`find`	Primeiro x	$O(N)$
`count`	Quantos x?	$O(N)$
`min` / `max`	Mínimo / máximo	$O(1)$
`min_element`	Iterador para o mínimo / máximo	$O(N)$
`accumulate`	Soma (ou redução) do range	$O(N)$

Atenção: `lower_bound` / `upper_bound` exigem container **ordenado**.

```cpp
vector<int> v = {1, 2, 3, 5, 5, 5, 7};

// Busca binária — requer vetor ordenado!
auto lo = lower_bound(v.begin(), v.end(), 5);
auto hi = upper_bound(v.begin(), v.end(), 5);

cout << *lo;          // 5  (1° ≥ 5)
cout << (hi - lo);    // 3  (quantos 5s)

bool tem = binary_search(v.begin(), v.end(), 3); // true

auto it = find(v.begin(), v.end(), 5); // Busca linear
if (it ≠ v.end()) cout << *it; // 5

int qtd = count(v.begin(), v.end(), 5); // 3

cout << min(3, 7);    // 3
cout << max(3, 7);    // 7

cout << *min_element(v.begin(), v.end()); // 1
cout << *max_element(v.begin(), v.end()); // 7


// Soma todos os elementos (init = 0)
```

Range-based `for`

`for (auto x : c)`: copia cada elemento; ok para `int` / `char`, evitar para objetos grandes.

`for (auto& x : c)`: referência; permite **modificar** elementos.

`for (const auto& x : c)`: referência constante; leitura sem cópia – **preferir** para `string`, `pair`, structs.

Funciona com qualquer container que tenha `begin()` / `end()`.

```
vector<int> v = {1, 2, 3, 4, 5};

// Cópia — ok para int
for (auto x : v)
    cout << x << ' ';

// Referência — modifica elementos
for (auto& x : v)
    x *= 2;  // v = {2, 4, 6, 8, 10}

// Const ref — leitura eficiente
for (const auto& x : v)
    cout << x << ' ';

// Pair em map — structured binding (C++17)
map<string, int> freq = {{"a", 1}, {"b", 2}};
for (const auto& [k, val] : freq)
    cout << k << ": " << val << '\n';
```

Passagem por referência

Por valor `f(vector<int> v)` – copia o container: $O(N)$. **Evitar.**

Por `const&` `f(const vector<int>& v)` – sem cópia, sem modificação. **Padrão para leitura.**

Por `&` `f(vector<int>& v)` – sem cópia, permite modificar in-place.

Regra prática: tipos primitivos (`int`, `char`, `bool`) por valor; containers e `string` por `const&` ou `&`.

```cpp
// Ruim — copia o vetor inteiro a cada chamada!
void imprime(vector<int> v) { ... }

// Leitura — sem cópia
int soma(const vector<int>& v) {
    int s = 0;
    for (const auto& x : v) s += x;
    return s;
}

// Modificação in-place — sem cópia
void dobrar(vector<int>& v) {
    for (auto& x : v) x *= 2;
}

int main() {
    vector<int> v = {1, 2, 3, 4, 5};
    cout << soma(v);  // 15
    dobrar(v);        // v = {2, 4, 6, 8, 10}
}
```

Qual container usar?

Precisa de…	Use
Lista com índice aleatório	`vector<T>`
Texto / sequência de chars	`string`
Tamanho fixo	`array<T,N>`
LIFO (pilha)	`stack<T>`
FIFO (fila)	`queue<T>`
Alteração duas pontas	`deque<T>`
Elementos únicos	`set<T>`
Contagem / chave→valor	`map<K,V>`
Lookup O(1)	`unordered_map<K,V>`
Maior/menor elemento	`priority_queue<T>`

Dúvida entre `map` **e** `unordered_map`**?** Prefira `map`: Por mais que unordered_map seja $O(1)$ médio, os setters tendem a forçar colisão, degradando pra $O(n)$. Use `unordered_map` somente se tiver certeza que não vai é possível forçar colisão.

Dúvida entre `vector` **e** `array`**?** Prefira `array` se souber o tamanho. `array` é mais rápido porq evita o overhead de realocação e é mais cache-friendly. Use `vector` só se o tamanho for desconhecido em compilation time.

Hora de codar

Centro de Informática UFPE · Universidade Federal de Pernambuco · MaratonaCIn

CF 4C – Registration System

O Codeforces processa n registros. Para cada nome de usuário:

- **Primeira ocorrência** → imprima `OK`
- **k-ésima ocorrência** ($k \geq 2$) → imprima `nome` + $(k-1)$

Constraints: $1 \leq n \leq 10^5$; nomes só com letras minúsculas.

Entrada

```
4
abacaba
acaba
abacaba
acab
```

Saída

```
OK
OK
abacaba1
OK
```

That's all, Folks!